

Imitation Learning for Generalizable Pick-and-Place Operations

Monish Piraimudi
mpiraimu@andrew.cmu.edu

Dan Jung
dtjung@andrew.cmu.edu

Abstract:

Pick-and-place operations in robotics are a cornerstone manipulation problem with broad relevance, from heavy industry to semiconductor fabrication. Traditional control methods rely heavily on hardcoded, scripted pipelines that are labor-intensive to configure and tune. This is not optimal for highly diverse manufacturing lines, therefore minimizing the complexity associated with training automation in industry is directly related to line productivity. The key optimization challenge is learning reliable control policies that generalize across varying tasks and endpoints with as few training episodes as possible. In this project, we address this challenge using imitation learning. We develop a full imitation learning pipeline to train the Emio-Labs robot to perform a pick and place operation. The pipeline is centered around an implicit behavioral cloning policy optimized via the Cross-Entropy Method during runtime. The policy is trained on data from expert demonstrations in the SOFA simulator. The policy is then validated in simulation and on the Emio-Labs robot where it was successful in transporting a cube between positions.

I. INTRODUCTION

A vast majority of modern industrial robotics fundamentally rely on manual programming to achieve automation for specific tasks. [1] Human-in-the-loop (HIL) programming is the process of configuring a robot through explicit instructions, relating to specific trajectories, joint configurations and process parameters, using teaching pendants, offline programming or teleoperation to successfully perform a task. This explicit instruction in HIL programming limits the functionality of robots due to registration of actions with specific workpiece geometries or configurations. Minor changes between product variants lead to all previous training becoming invalid, requiring labor intensive HIL retuning which is not ideal in high diversity manufacturing lines. [2]

Pick and place (PnP) operations are an important example of HIL programming being a major bottleneck. PCB assembly and semiconductor packaging are examples of diverse manufacturing lines that require frequent reprogramming due to rapid product iteration cycles. [3] Methods that enable PnP systems to generalize to new task variations without full

re-teaching are highly desirable, as they reduce overhead and improve line productivity. [4]

Current methods for control in manufacturing robotics include model based controllers. [5] Model based controllers, such as PID and MPC. In PID you are tuning KI, KP, and KD terms and control is based on a deviation metric $e(t)$.

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt}$$

In MPC, the optimization is performed over a finite horizon T by selecting a sequence of control inputs $u_1, u_2, \dots, u_T$ that minimize a cumulative cost. The cost function $\ell(x_t, u_t)$ penalizes deviation of the state x_t .

$$\min_{u_{0:T}} \sum_{t=0}^T \ell(x_t, u_t)$$

However, model-based controllers rely heavily on the accuracy of their modelled state dynamics. They require frequent recalibrations when process conditions change. Model based controllers generalize only within narrow operational envelopes and do not scale efficiently to high variability manufacturing environments. [6] Mansouri et al. [7] in their study go over these limitations and discuss how task and motion planning methods in industrial robotics rely on explicit domain models and assumptions about task structure and environment dynamics, which must be updated when operating conditions change. While not related to industrial robotics, Jiang et al. [8] in their study discuss the limitations of advanced optimization-based controllers such as MPC depending on accurate system models and constraint formulations, which degrade under modeling errors and require retuning under environmental variation.

Learning-based control methods improve upon model-based approaches by learning system dynamics directly from data. They generalize within the training distribution and are more robust to process variations due to neural networks capturing highly nonlinear relationships between process parameters. These large-scale non-convex models are typically optimized using gradient-based methods such as mini-batch

SGD and ADAM. Broadly, learning based methods are split into reinforcement and supervised learning.

Deep reinforcement learning (DRL) is one approach to learn high nonlinear relationships in manufacturing actions for PnP robots. [9]

$$\pi(u | x) = \max_{\pi} \mathbb{E} \left[\sum_{t=0}^T r_t \right]$$

$$Q(x, u) = r(x, u) + \gamma \mathbb{E}_{x'} \left[\max_{u'} Q(x', u') \right]$$

In DRL, the goal is to learn a policy π , which maps a state x to an action u , such that the expected cumulative reward r_t over a time horizon T is maximized. This is achieved through repeated interaction with the environment and is formally captured using the Bellman equation for the Q-function. It measures quality of action u in state x followed by optimal behavior. It defines the immediate reward $r(x, u)$, a discount factor γ that controls future rewards, an expectation $\mathbb{E}_{x'}[\cdot]$ over the next state x' , and $\max_{u'} Q(x', u')$ which selects the best future action.

$$\mathcal{D} = \{(x_i, u_i)\}_{i=1}^N \quad f_{\theta}(u) \approx x$$

$$\min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(u_i), x_i)$$

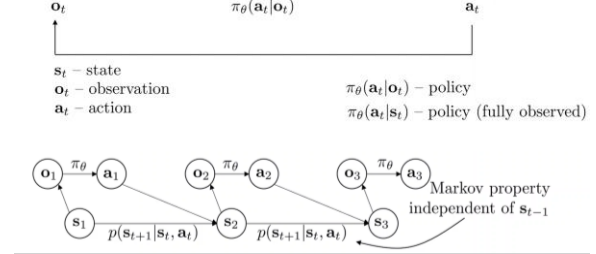
Another approach in learning-based control is supervised learning methods. In supervised learning, models learn from structured training dataset D by finding a function f with parameters θ that maps a state x to an action u . Optimization here consists of learning θ to minimize the average training loss.

DRL models are the current paradigm in learning-based control but they can be less practical for manufacturing PnP systems due to its relative sensitivity to reward design, and potential challenges in achieving stable, efficient learning under real-world operational constraints. On the other hand, supervised learning is a predecessor technology and typically has very large data requirements and performs optimally with highly structured datasets, the generation of which is labor intensive. However, we make the assumption that manufacturing lines have a broad corpus of historic operational data readily available, in which case supervised learning is a straight forward approach, especially for simple PnP robotics.

Therefore, we selected Behavioral cloning (BC). BC is an imitation learning method that learns a policy over expert demonstrations. Given a dataset $D = \{(x_i, u_i)\}$ of states x_i and expert actions u_i , BC learns a parameterized policy $\pi_{\theta}(u | x)$ by solving:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{(x,u) \sim D} [\mathcal{L}(\pi_{\theta}(x), u)]$$

where $\mathcal{L}$ is usually a MSE regression loss. By learning the function, BC directly imitates the expert's mapping from observed states to actions, without requiring a reward signal or interaction with the environment during training. [10]



However, traditional BC methods suffer from error drift. The diagram above illustrates a Partially Observable Markov Decision Process. It shows how information flows through time. We can see how state s_1 informs observation o_1 which is then mapped to action a_1 by learned policy π_{θ} . Action a_1 then determines state s_2 and so on. The issue is the independence of future states from past states. This is called the Markov property. If the agent makes a small mistake, it may enter any state s the expert never visited, and since it only learned from expert observations, it cannot recover. All further states lead to drift away from the desired behavior.

One solution to this problem in BC was proposed by Florence et al. [11] at Google Robotics. In their paper they describe an implicit policy defined through an Energy-based model. A standard form for the implicit policy is presented below:

$$\pi(s) = \arg \min_a E_{\theta}(s, a)$$

An implicit policy helps mitigate drift in BC by learning a conditional distribution $p_{\theta}(a | s)$ (via the energy-based formulation) rather than a single deterministic mapping. The model does not predict an action directly, instead it evaluates many possible actions, selecting the one with least energy. This prevents issues where a single deterministic mapping (as in explicit policies) may lead to averaging of multiple valid actions into a potentially invalid one. Whereas in an implicit policy, the policy is more robust in off-distribution states. When small errors occur, the model can still select a plausible action from a set of valid behaviors rather than committing to an averaged, incorrect action. This is possible because the implicit policy represents multiple valid actions as distinct minima in the energy landscape, allowing it to select among them rather than collapsing them into an invalid average. Beyond robustness to drift, this is particularly applicable to cobots, as the implicit policy can re-evaluate and select among multiple valid action minima in real time.

An implicit policy learns by assigning low energy (high score) to correct expert actions and higher energy to incorrect ones, which induces a conditional distribution over actions. Training is done via contrastive loss by comparing positive (expert) and negative (non-expert) actions. This is equivalent to maximizing the log-likelihood of expert actions under the model, or minimizing the negative log-likelihood:

$$E_{\theta}(s, a^*) < E_{\theta}(s, a^-) \quad \min_{\theta} - \sum_{i=1}^N \log p_{\theta}(a_i^* | s_i)$$

Here, cross-entropy loss arises naturally as the negative log-likelihood of the true (expert) action under this distribution, effectively pushing the model to decrease the energy of positive examples while increasing the energy of negative ones. In this sense, the model is trained to answer “which action is correct?” by assigning the highest probability (lowest energy) to expert actions across all training examples.

Coming back to our problem, in implicit BC, the presence of multiple distinct minima in the energy landscape allows the model to represent several valid grasps, placements, and motion strategies simultaneously; this enables PnP robots to generalize to novel configurations.

This is the value of implicit BC and is at the heart of our motivation to implement this in our project. This mechanism of learning a manifold of valid actions instead of learning the correct output makes implicit BC more generalizable than standard explicit BC.

In this project, we implement an implicit policy optimized behavioral cloning pipeline for the Emio Labs robot to perform PnP operations. The workflow consists of defining an expert simulation environment in SOFA, generating expert demonstration data within SOFA, and validating the learned policy in simulation before deployment on the physical robot platform. Finally, the full codebase is being packaged into an Emio Labs module for documentation and pedagogical use.

II. PROBLEM STATEMENT

The objective of the project is to develop a control framework for generalizable PnP operations on the Emio robot platform that can generalize across variations in parts, placement, configuration, and task instantiation without requiring HIL input. The target domain is industrial robotics, particularly PnP automation and robotics in electronics and semiconductor packaging industries (such as SCARA robots). We assume that data intended for expert demonstrations are readily available from existing historical corpus, teleop or simulation.

We require that the learned policy must be transferable to a real robotic system (Emio in this case). We are noting that for the current project we are not focusing on all possible variations in parts, placements or configurations. We simplify the problem and the training of the policy to a simple predetermined location and movement path.

The optimization problem is inherently non-convex. Therefore, we are using a neural network to parameterize the energy function. The problem is also continuous because we assume that both the observations and actions are continuous valued (to machine precision) and are not discrete. Since the model learns allowed and disallowed actions from the data of expert demonstration, the problem is also implicitly constrained by the data.

Some of the main challenges to this optimization problem are as follows:

1. **Multiple valid actions:** For any given state, the implicit BC function can have multiple minimas, hence multiple valid actions. This needs to be represented and collapsed into a single action during runtime.
2. **Training transferability from SOFA to real world:** Since we are training to deploy on the real robot, we need to ensure that SOFA expert demonstrations are of high enough quality to transfer into real world execution.
3. **Data diversity:** Implicit BC benefits from a highly diverse dataset of expert demonstrations. Therefore, we need to introduce diversity in our training data, however limited.
4. **Inference time:** Actions require solving a highly non-convex optimization problem at inference, meaning we need to make sure the algorithm choices are efficient and stable.

III. METHODS

The project is organized around one central SOFA scene which is the source of truth for robot and task geometry, expert task execution, etc. We implemented each stage of the training workflow as separate .py files but they all reuse the central scene file.

Our stack combines the following:

1. SOFA for robot simulation and expert demonstrations
2. Emio/Compliance Lab components for connecting to real world robot
3. PyTorch for policy training and inference

The implemented task is a single-cube pick-and-place problem and the expert demonstrations within the scene are as follows:

1. a cube starts on a tray

2. the robot must approach it, grasp it, lift it, move it to a fixed target, release it, and retreat
3. the cube start position is randomized in a continuous workspace, with default bounds x in $[-35, 10]$ mm and z in $[-30, 20]$ mm

We divided the control logic into scripted phases so only the motor coordinates are learnt:

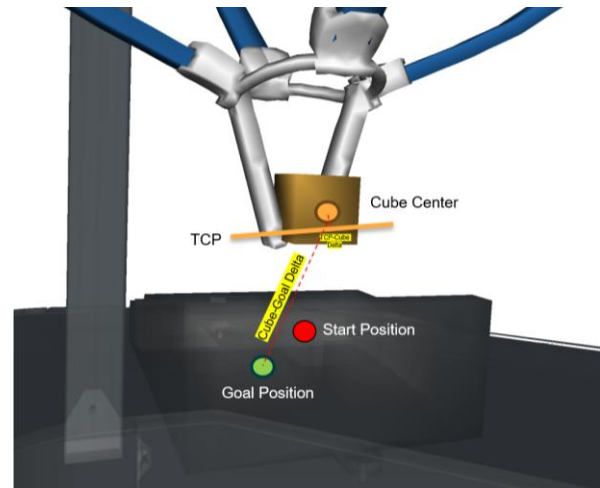
1. Approach to pick
2. Descend to pick
3. Close gripper
4. Lift
5. Approach to place
6. Descend to place
7. Open gripper
8. Retreat

To determine if a gripping action was successful in attaching to the cube, we used a simple threshold-based objective instead of manually modelling the physics interactions between gripper and cube at the cost of some accuracy:

1. the cube is kept kinematic
2. grasp is declared when the cube is within an attachment threshold of the tool center and the gripper opening is sufficiently closed
3. once attached, the evaluator directly moves the cube with the tool center point using a fixed offset
4. lift success is declared when the cube rises above its initial tray height by a configured amount
5. release is detected when the gripper reopens
6. after release, the cube is dropped vertically back to tray height and then marked as placed

Initially we wanted to use the RGB camera onboard the Emio Labs robot to do pose estimation and tracking. But we resorted to purely state based control and did not include RGB data. The data required to fully define a state at any time point is:

- a 17D geometric state vector
- a 4D motor-angle action
- executed action after clipping
- Performance metrics:
 - Phase index: current task stage
 - Object pose: cube position orientation
 - Target pose: desired placement poses
 - TCP pose: end-effector reference pose
 - Gripper state: open-close holding status
 - Success flags: task completion indicators



The 17D geometric state vector is defined as follows (shown above):

- TCP position (x,y,z)
 - midpoint of common axis between gripper finger tips
- cube position (x,y,z)
- goal position (x,y,z)
- TCP-to-cube delta (x,y,z)
- cube-to-goal delta (x,y,z)
- normalized gripper opening
- held flag

The 4D action vector is the absolute motor angles, one for each actuator. The policy is responsible only for the angle of the motors, everything else is scripted and was included to resolve runtime/logical errors and simplify training. This leaves us with a runtime environment that has:

1. high level sequencing of tasks and gripper timing that is scripted
2. low level motor angle selection learnt by policy

Expert demonstration is stored as episodes. Each demonstration is slightly randomized in based on a size that determined certain parameters within a limited window for variation. For each seed:

- a cube pose is sampled in the workspace
- the pick point is aligned to that cube pose
- the place target stays fixed unless overridden
- the expert controller is rolled out phase by phase
- the full trajectory is logged to a compressed. npz episode file

The dataset stores at least 100 episodes before training the implicit BC controller. The network architecture used for behavioral cloning is as follows:

- State encoder: $17 \rightarrow 64 \rightarrow 64$
- Action encoder: $4 \rightarrow 128 \rightarrow 128$
- Energy head: $(64 + 128) \rightarrow 256 \rightarrow 1$

The state encoder maps the 17D state into a learned latent state embedding. The action encoder maps a candidate 4D motor action into a learned action embedding. Finally, the energy head concatenates both embeddings and outputs one scalar energy $E(s,a)$.

Training uses contrastive-style classification:

- each expert action is the positive sample
- 63 negative actions are generated per positive sample
- negatives are a 50/50 mix of globally uniform samples inside action bounds and local perturbations around the expert action
- the loss is cross-entropy over negative energies, pushing expert actions to lower energy than sampled non-expert actions

We are using two different optimization layers separated by training and inference stages in our pipeline. We use mini-batch training implemented through PyTorch's DataLoader with shuffling. Model parameters are optimized using the ADAM optimizer. During inference, we use the Cross-Entropy Method (CEM) which is a derivative-free sampling algorithm (similar to a Monte Carlo sampler). Since the policy defines an energy landscape over actions, CEM is an efficient way to search that landscape for low-energy (high-quality) actions. CEM is particularly well-suited for implicit policies because it performs population-based sampling over the action space, allowing it to efficiently explore and locate multiple low-energy minima, whereas gradient-based methods tend to converge to a single local minimum and can miss other valid modes in a multi-modal energy landscape. It searches for a low-energy action using a CEM:

- 192 samples per iteration
- 24 elites
- 4 CEM iterations
- minimum search std 0.05

To ensure the validity of our dataset/controller, we implemented an evaluator for the policy that can assign metric to each episode based on the following rollout metrics:

- pick_success
- place_success
- total_success
- final_place_error_mm
- dropped_object_rate

- failure_phase_counts

IV. RESULTS

A. Software

To ensure you are able to run all the steps in the emio-labs module as intended, please run the install dependencies tab and restart your terminal. If that does not work, please follow the below commands to manually run each module assuming SOFA is installed and works on your system:

- `python -m pip install -r requirements.txt`
- `python collect_il_dataset.py --episodes 100 --max-attempts 140 --workspace-bounds-mm -35 10 -30 20 --save-failed-episodes`
- `python train_il_policy.py --dataset-dir data/results/il_pick_place/episodes --output-path data/results/il_pick_place/bc_policy.pth`
- `python evaluate_il_policy.py --mode policy --policy-path data/results/il_pick_place/bc_policy.pth --workspace-bounds-mm -35 10 -30 20`

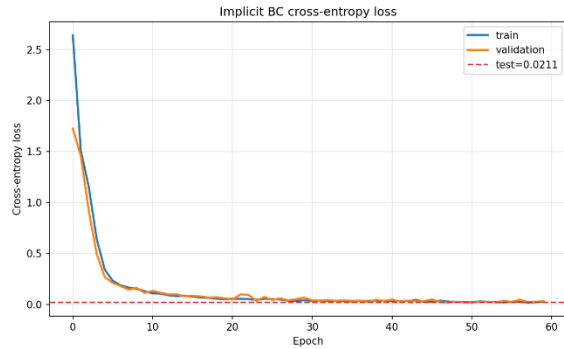
B. Simulation Results

Table 1: Summary of results

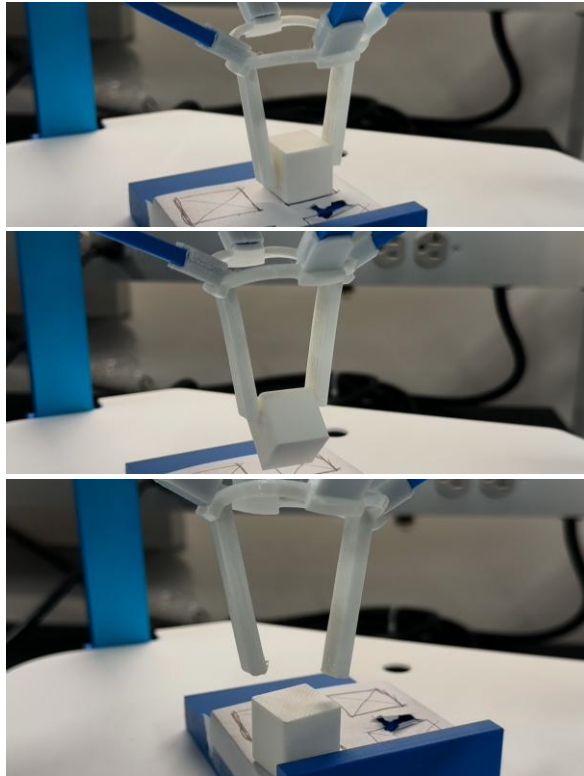
Metric	Value
Saved expert episodes	100
Training / test episode split	80 / 10 / 10
Training / test timesteps	4386 / 557 / 551
Best validation loss	0.0192
Test loss	0.0211
Pick success rate	100%
Place success rate	100%
Total success rate	100%
Median final place error	5.671 mm

Table 1 summarizes the quantitative results obtained from the training pipeline. The results show that the policy successfully learns the expert behavior. The closed-loop performance matches that of the expert. The training and test losses are low, which indicates that the model can fit the demonstrated state-action mapping. However, the median final place error is 5.67mm which is relatively high for a fully accurate policy. This is due to inaccuracies in the SOFA simulation environment, combined with simplifications in the expert setup (e.g., no collisions or contact modelling between gripper and cube), which led to high final placement errors in the expert demonstrations that were subsequently learned by the trained policy.

The implicit BC loss curve (below) shows that the implicit behavior cloning policy trains efficiently and converges



quickly. Both the training and validation cross-entropy losses drop sharply in the first few epochs, from roughly the 1.5-2.6 range down to near zero by about 10-15 epochs. This shows us that the model is able to learn the expert state-action structure efficiently from the demonstration dataset. The plateau following the drop is indication that the optimization problem is well behaved. The real-world demo pictures are shown below (pick, transport, place):



V. CONCLUSIONS

We were able to successfully implement a complete implicit imitation learning pipeline for the Emio robot. We were able to integrate expert demonstration, dataset collection, policy training, and closed-loop evaluation within one reproducible emio-labs module. The learned implicit behavior-cloning policy successfully captures the expert's motion, and the

training results show strong performance with low validation and test loss.

Closed-loop performance matches the scripted expert baseline. Although the model learns the state-action relationship, the median final place error being 5.67mm shows that the quality in the training dataset is directly tied to the quality of policy trained.

Future work could improve robustness by collecting more diverse demonstrations, including failure and recovery cases, expanding to vision-based policy learning, and reducing reliance on scripted sequencing. Extending the method toward real hardware execution would also require better contact modeling, calibration, and sim-to-real robustness.

VI. AUTHORSHIP ATTRIBUTION

Dan Jung: Implementation of implicit learning policy and the emio-labs module

Monish: Development of evaluation framework, Presentation and Report writing

REFERENCES

- [1] Merlo E, Lagomarsino M and Ajoudani A 2025 A Human-in-the-loop Approach to Robot Action Replanning through LLM Common-Sense Reasoning
- [2] Wang B, Tao F, Fang X, Liu C, Liu Y and Freiheit T 2021 Smart Manufacturing and Intelligent Manufacturing: A Comparative Review *Engineering* **7** 738–57
- [3] Yu Z, Zhang P and Shi J 2026 Transformation of industrial robotics with natural language models: Recent progress and future prospects *Robotics and Computer-Integrated Manufacturing* **97** 103113
- [4] Yoshimura K, Yamauchi Y and Takahashi H 2025 Managing the variability of a logistics robotic system *Journal of Systems and Software* **222** 112328
- [5] Zhao Z, Cheng S, Ding Y, Zhou Z, Zhang S, Xu D and Zhao Y 2024 A Survey of Optimization-based Task and Motion Planning: From Classical To Learning Approaches
- [6] Yang S, Pei C, Dai R and Sun C 2024 Towards an Adaptable and Generalizable Optimization Engine in Decision and Control: A Meta Reinforcement Learning Approach
- [7] Mansouri M, Pecora F and Schüller P 2021 Combining Task and Motion Planning: Challenges and Guidelines *Front. Robot. AI* **8** 637888
- [8] Jiang Z, Wang Y, Wang S, Bi S and Chen J 2024 Motion Planning and Control with Environmental Uncertainties for Humanoid Robot *Sensors* **24** 7652
- [9] Shiyu F 2025 Reinforcement learning-driven deep learning approaches for optimized robot trajectory planning *Sci Rep* **15** 37898
- [10] Robey A, Hu H, Lindemann L, Zhang H, Dimarogonas D V, Tu S and Matni N 2020 Learning Control Barrier Functions from Expert Demonstrations
- [11] Florence P, Lynch C, Zeng A, Ramirez O, Wahid A, Downs L, Wong A, Lee J, Mordatch I and Tompson J 2021 Implicit Behavioral Cloning